

NAME:-Ananya manish jadhav

PRN:-202401040097

ROLL NO:-62

INSERTION

CODE:-

```
#include <iostream>
```

```
#include <stack>
```

```
using namespace std;
```

```
struct Node {
```

```
    int data;
```

```
    Node* left;
```

```
    Node* right;
```

```
    int height;
```

```
};
```

```
int height(Node* n) {
```

```
    if (n == NULL)
```

```
        return 0;
```

```
    return n->height;
```

```
}
```

```
int getBalance(Node* n) {
```

```
    if (n == NULL)
```

```
        return 0;
```

```
    return height(n->left) - height(n->right);
```

```
}
```

```
Node* createNode(int data) {  
  
    Node* newNode = new Node();  
  
    newNode->data = data;  
  
    newNode->left = newNode->right = NULL;  
  
    newNode->height = 1;  
  
    return newNode;  
  
}
```

```
Node* rightRotate(Node* y) {  
  
    Node* x = y->left;  
  
    Node* T2 = x->right;  
  
  
    x->right = y;  
  
    y->left = T2;  
  
  
    y->height = max(height(y->left), height(y->right)) + 1;  
    x->height = max(height(x->left), height(x->right)) + 1;  
  
  
    return x;  
  
}
```

```
Node* leftRotate(Node* x) {  
  
    Node* y = x->right;  
  
    Node* T2 = y->left;  
  
  
    y->left = x;
```

```
x->right = T2;
```

```
x->height = max(height(x->left), height(x->right)) + 1;
```

```
y->height = max(height(y->left), height(y->right)) + 1;
```

```
return y;
```

```
}
```

```
Node* insert(Node* node, int data) {
```

```
    if (node == NULL)
```

```
        return createNode(data);
```

```
    if (data < node->data)
```

```
        node->left = insert(node->left, data);
```

```
    else if (data > node->data)
```

```
        node->right = insert(node->right, data);
```

```
    else
```

```
        return node;
```

```
    node->height = 1 + max(height(node->left), height(node->right));
```

```
    int balance = getBalance(node);
```

```
    if (balance > 1 && data < node->left->data)
```

```
        return rightRotate(node);
```

```
    if (balance < -1 && data > node->right->data)
```

```
    return leftRotate(node);

    if (balance > 1 && data > node->left->data) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }

    if (balance < -1 && data < node->right->data) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}
```

/* Recursive Traversals */

```
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}
```

```
void preorder(Node* root) {
    if (root != NULL) {
```

```
    cout << root->data << " ";

    preorder(root->left);

    preorder(root->right);

}

}
```

```
void postorder(Node* root) {

    if (root != NULL) {

        postorder(root->left);

        postorder(root->right);

        cout << root->data << " ";

    }

}
```

/* Non Recursive Inorder */

```
void nonRecInorder(Node* root) {

    stack<Node*> s;

    Node* curr = root;

    while (curr != NULL || !s.empty()) {

        while (curr != NULL) {

            s.push(curr);

            curr = curr->left;

        }

        curr = s.top();

    }

}
```

```
s.pop();

cout << curr->data << " ";

curr = curr->right;

}

}
```

/* Non Recursive Preorder */

```
void nonRecPreorder(Node* root) {

    if (root == NULL)

        return;

    stack<Node*> s;

    s.push(root);

    while (!s.empty()) {

        Node* temp = s.top();

        s.pop();

        cout << temp->data << " ";

        if (temp->right)

            s.push(temp->right);

        if (temp->left)

            s.push(temp->left);

    }
```

```
}
```

```
/* Non Recursive Postorder */
```

```
void nonRecPostorder(Node* root) {
```

```
    if (root == NULL)
```

```
        return;
```

```
    stack<Node*> s1, s2;
```

```
    s1.push(root);
```

```
    while (!s1.empty()) {
```

```
        Node* temp = s1.top();
```

```
        s1.pop();
```

```
        s2.push(temp);
```

```
        if (temp->left)
```

```
            s1.push(temp->left);
```

```
        if (temp->right)
```

```
            s1.push(temp->right);
```

```
    }
```

```
    while (!s2.empty()) {
```

```
        cout << s2.top()->data << " ";
```

```
        s2.pop();
```

```
    }
```

```
}
```

```
int main() {  
  
    Node* root = NULL;  
  
    int n, x;  
  
    cout << "Enter number of nodes: ";  
  
    cin >> n;  
  
    cout << "Enter elements:\n";  
    for (int i = 0; i < n; i++) {  
        cin >> x;  
        root = insert(root, x);  
    }  
  
    cout << "\nRecursive Inorder: ";  
    inorder(root);  
  
    cout << "\nRecursive Preorder: ";  
    preorder(root);  
  
    cout << "\nRecursive Postorder: ";  
    postorder(root);  
  
    cout << "\nNon Recursive Inorder: ";  
    nonRecInorder(root);  
  
    cout << "\nNon Recursive Preorder: ";
```

```
nonRecPreorder(root);

cout << "\nNon Recursive Postorder: ";

nonRecPostorder(root);


return 0;

}
```

INPUT:-

Enter number of nodes: 5

Enter elements:

10 20 30 40 50

OUTPUT:-

Recursive Inorder: 10 20 30 40 50

Recursive Preorder: 30 20 10 40 50

Recursive Postorder: 10 20 50 40 30

Non Recursive Inorder: 10 20 30 40 50

Non Recursive Preorder: 30 20 10 40 50

Non Recursive Postorder: 10 20 50 40 30